



WESTERN MICHIGAN
UNIVERSITY

College of Engineering and Applied Sciences

Course Name: Lab. Intro. To Microprocessors

Course Number: ECE 2510

Instructors:

[Christian Blanchard](#)

[Pravana Avuthu](#)

Project Report

Students:

Wang, Avery – WIN: 175628745

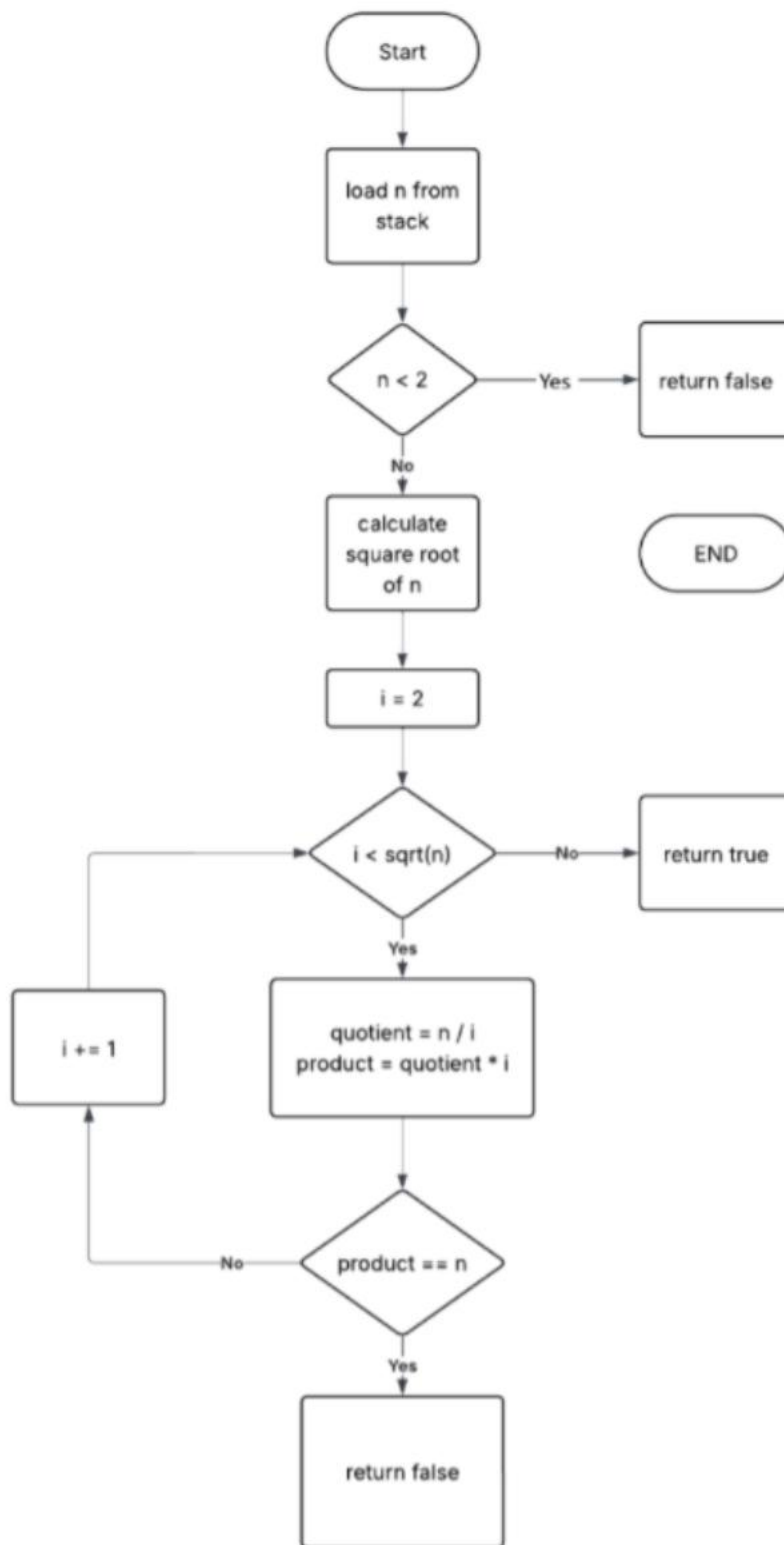
De Leon Santos, Sebastian Manuel – WIN: 577866168

Description of the project

This project uses a Nucleo board STM32L476RGT soldered to a Printed Circuit Board connected with a 4-digit 7 segment display in Port B. We were tasked in creating an arm assembly code for the board that iterates through an array of numbers, and displays the prime numbers found in the array using the display.

Task 1.

Flowchart:



Stack frame:

SP + 28	Result space. Pushed from R1 in main
SP + 24	Input parameter. Pushed from R0 in main
SP + 20	LR
SP + 16	R8
SP + 12	R7
SP + 8	R6
SP + 4	R5
SP + 0	R4 (stack pointer is here during the isPrime subroutine)

Code:

```
C:\Users\sebas\wmu_classes\spring2026\intro_microprocessors\lab\ece2510_final_project\final_project.uvprojx - µVision [Non-Commercial Use Lice...
File Edit View Project Flash Debug Peripherals Tools SVCS Window Help
ldr
task1.s startup_stm32i476xx.s
1      AREA    main, CODE, READONLY
2      EXPORT  __main
3      ENTRY   __main
4
5      __main   PROC
6
7          ldr    r0, =7
8          ldr    r1, =0
9          push   {r0, r1}
10         bl     isPrime
11
12         pop    {r0, r1}      ; R1 now holds the result
13
14     stop     b     stop
15     ENDP
16
17
18     ; isPrime Function
19     ; -----
20     isPrime   PROC
21         push   {r4-r8, lr}    ; sp drops by 24
22
23         ldr    r4, [sp, #24]   ; load n from stack
24
25         ldr    r5, =2
26         cmp    r4, r5         ; if n < 2 return false
27         blt    return_false   ;
28
29         ; SQRT FUNCTION
30         ; -----
31
32         mov    r6, #0
33     calc_sqrt mul    r7, r6, r6
34         cmp    r7, r4
35         bgt    sqrt_found
36
37         add    r6, r6, #1
38         b      calc_sqrt
39     sqrt_found sub    r6, r6, #1
40
41
42     prime_loop cmp    r5, r6      ; r5 = i (counter) r6 = sqrt(n) returned by sqrt f
43         bgt    return_true      ; if at the end of loop, return true
44
45         udiv   r7, r4, r5        ; r7 = quotient = n / i
46         mul    r8, r7, r5        ; r8 = product = quotient * i
47
48         cmp    r8, r4           ; if product = n return false
49         beq    return_false
50
51         add    r5, r5, #1        ; i += 1
52         b      prime_loop
53
54
55
56     return_true ldr    r1, =1
57         str    r1, [sp, #28]
58         pop    {r4-r8, pc}
59
60     return_false ldr    r1, =0
61         str    r1, [sp, #28]
62         pop    {r4-r8, pc}
63     ENDP
Simulation
```

Cycles taken for task 1 for n = 7:

C:\Users\sebas\wmu_classes\spring2026\intro_microprocessors\lab\ece2510_final_project\final_project.uvprojx - µVision [Non-Commercial]

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help

Registers Disassembly

Register	Value
R1	0x00000001
R2	0x00000000
R3	0x00000000
R4	0x00000000
R5	0x00000000
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x20000600
R14 (LR)	0x080001F3
R15 (PC)	0x080001F4
xPSR	0x21000000
N	0
Z	0
C	1
V	0
Q	0
GE	0x0
T	1
IT	Disabled
ISR	0
Banked	
MSP	0x20000600
PSP	0x00000000
System	
BASEPRI	0x00
PRIMASK	0
FAULTMASK	0
CONTROL	0x00
Internal	
Mode	Thread
Privilege	Privileged
Stack	MSP
States	101
Sec	0.00000842
FPU	

```

14: stop          b      stop
15:
16:
17:
18: ;      isPrime Function
19: ;      -----
20: isPrime      PROC

task1.s startup_stm32i476xx.s
4
5  _main          PROC
6
7      ldr        r0, =7
8      ldr        r1, =0
9      push       {r0, r1}
10     bl         isPrime
11
12     pop        {r0, r1}      ; R1 now holds
13
14 stop          b      stop
15
16 ENDP
17
18 ;      isPrime Function
19 ;      -----
20 isPrime      PROC
21     push       {r4-r8, lr}   ; sp drops by
22
23     ldr        r4, [sp, #24] ; load n from
24
25     ldr        r5, =2
26     cmp        r4, r5        ; if n < 2 ret
27     blt        return_false ;
28
29 ;      Sqrt Function
30 ;      -----
31
32 calc_sqrt     mov        r6, #0
33               mul        r7, r6, r6
34               cmp        r7, r4
35               bgt        sqrt_found

```

Command

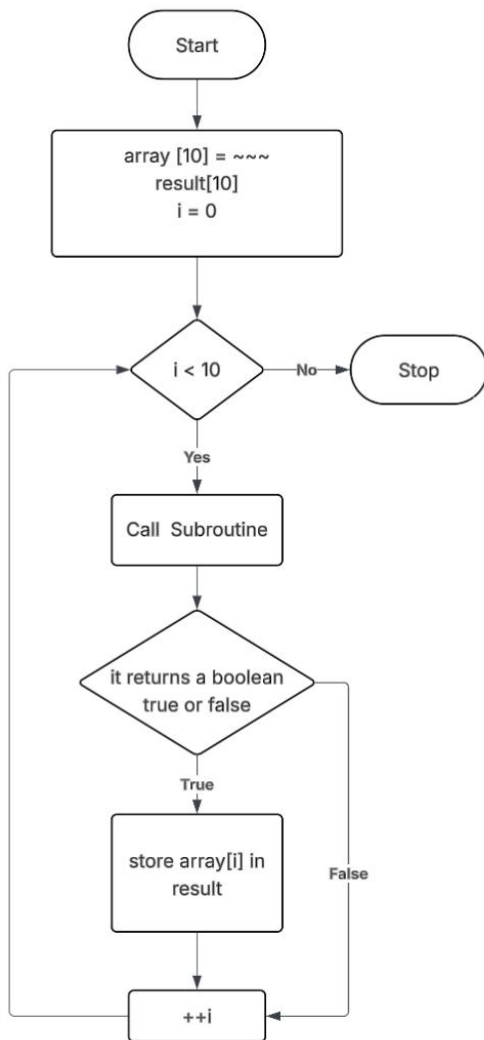
Running with Code Size Limit: 32K
Load "C:\\Users\\sebas\\wmu_classes\\spring2026\\intro_microprocessors\\lab\\ece2510_final_project\\final_project.uvprojx"
BS \\final_project\\task1.s\\14

Call Stack + Locals

Name	Location/Value
..... _main	0x080001F4

Task 2

Flow Chart



Stack frame:

SP + 28	Result space. Pushed from R1 in main
SP + 24	Input parameter. Pushed from R0 in main
SP + 20	LR
SP + 16	R8
SP + 12	R7
SP + 8	R6
SP + 4	R5
SP + 0	R4 (stack pointer is here during the isPrime subroutine)

Code:

```
1  AREA    mydata, DATA, READONLY
2
3  array   DCD    3,5,7,9999,20,23,100,97,98,90
4
5  AREA    mybutt, DATA, READWRITE
6
7  result  SPACE  40
8
9
10 AREA    main, CODE, READONLY
11 EXPORT  __main
12 ENTRY   __main
13
14 ;===== MAIN =====
15
16 __main   PROC
17
18         mov     r0, #0           ; n
19         mov     r1, #0           ; true or false
20         mov     r2, #0           ; counter i = 0
21         ldr     r12, =array
22         ldr     r11, =result
23
24 loop
25         ldr     r0, [r12, r2, LSL#2] ; load array value into r0, register offset 4 bytes
26         push    {r0, r1}
27         bl      isPrime
28         pop     {r0, r1}         ; R1 now holds if true or false
29
30         cmp     r1, #1           ; if true store
31         beq     true
32         b       false
33 true
34         str     r0, [r11], #4
35 false
36         add     r2, #1           ; ++i
37         cmp     r2, #10
38         blt     loop
39
40 stop
41         b       stop
42         ENDP
43
44 ; isPrime Function
45 ; -----
46 isPrime  PROC
47         push    {r4-r8, lr}     ; sp drops by 24
48         ldr     r4, [sp, #24]    ; load n from stack
49
50         ldr     r5, =2           ;
51         cmp     r4, r5           ; if n < 2 return false
52         blt     return_false    ;
53
54 ; Sqrt Function
55 ; -----
56 calc_sqrt
57         mov     r6, #0
58         mul     r7, r6, r6
59         cmp     r7, r4
60         bgt     sqrt_found
61
62         add     r6, r6, #1
63         b       calc_sqrt
64 sqrt_found
65         sub     r6, r6, #1
66
67 prime_loop
68         cmp     r5, r6           ; r5 = i (counter) r6 = sqrt(n) returned by sqrt function
69         bgt     return_true     ; if at the end of loop, return true
70
71         udiv    r7, r4, r5       ; r7 = quotient = n / i
72         mul     r8, r7, r5       ; r8 = product = quotient * i
73
74         cmp     r8, r4           ; if product = n return false
75         beq     return_false
76
77         add     r5, r5, #1       ; i += 1
78         b       prime_loop
79
80 return_true
81         ldr     r1, =1
82         str     r1, [sp, #28]
83         pop     {r4-r8, pc}
84
85 return_false
86         ldr     r1, =0
87         str     r1, [sp, #28]
88         pop     {r4-r8, pc}
89         ENDP
```


Cycles taken for task2 when array[10] = {3,5,7,9999,20,23,100,97,98,90};

The screenshot displays the Keil uVision IDE interface. On the left, the Register window shows the state of Core registers R0 through R15 and the xPSR register. The main window displays the assembly code for 'task3.s', which includes the 'isPrime' function definition. The right pane shows the Command window with the load command and the Memory window showing the address 0x20000000.

Register Window:

Register	Value
R0	0x0000005A
R1	0x00000000
R2	0x0000000A
R3	0x00000000
R4	0x00000000
R5	0x00000000
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x20000014
R12	0x08000270
R13 (SP)	0x20000628
R14 (LR)	0x08000203
R15 (PC)	0x08000216
xPSR	0x61000000

Assembly Code (task3.s):

```

28
29      cmp     r1, #1
30      beq     true      ; if true store
31      b       false
32      true    str     r0, [r1], #4
33      false
34      add     r2, #1      ; ++i
35      cmp     r2, #10
36      blt     loop
37
38      stop    b       stop
39      ENDP
40
41
42      ; isPrime Function
43      ; -----
44      isPrime PROC
45      push    {r4-r8, lr} ; sp drops by 24
46
47      ldr     r4, [sp, #24] ; load n from stack
48
49      ldr     r5, =2      ;
50      cmp     r4, r5      ; if n < 2 return false
51      blt     return_false ;
52
53      ; SQRT FUNCTION
54      ; -----
55
56      mov     r6, #0
57      calc_sqrt mul    r7, r6, r6
58      cmp     r7, r4
59      bgt     sqrt_found
60
61      add     r6, r6, #1

```

Command Window:

```

Running with Code Size Limit: 32K
Load "C:\\Everything Else\\MicroP Keil\\Final Project\\Ob

```

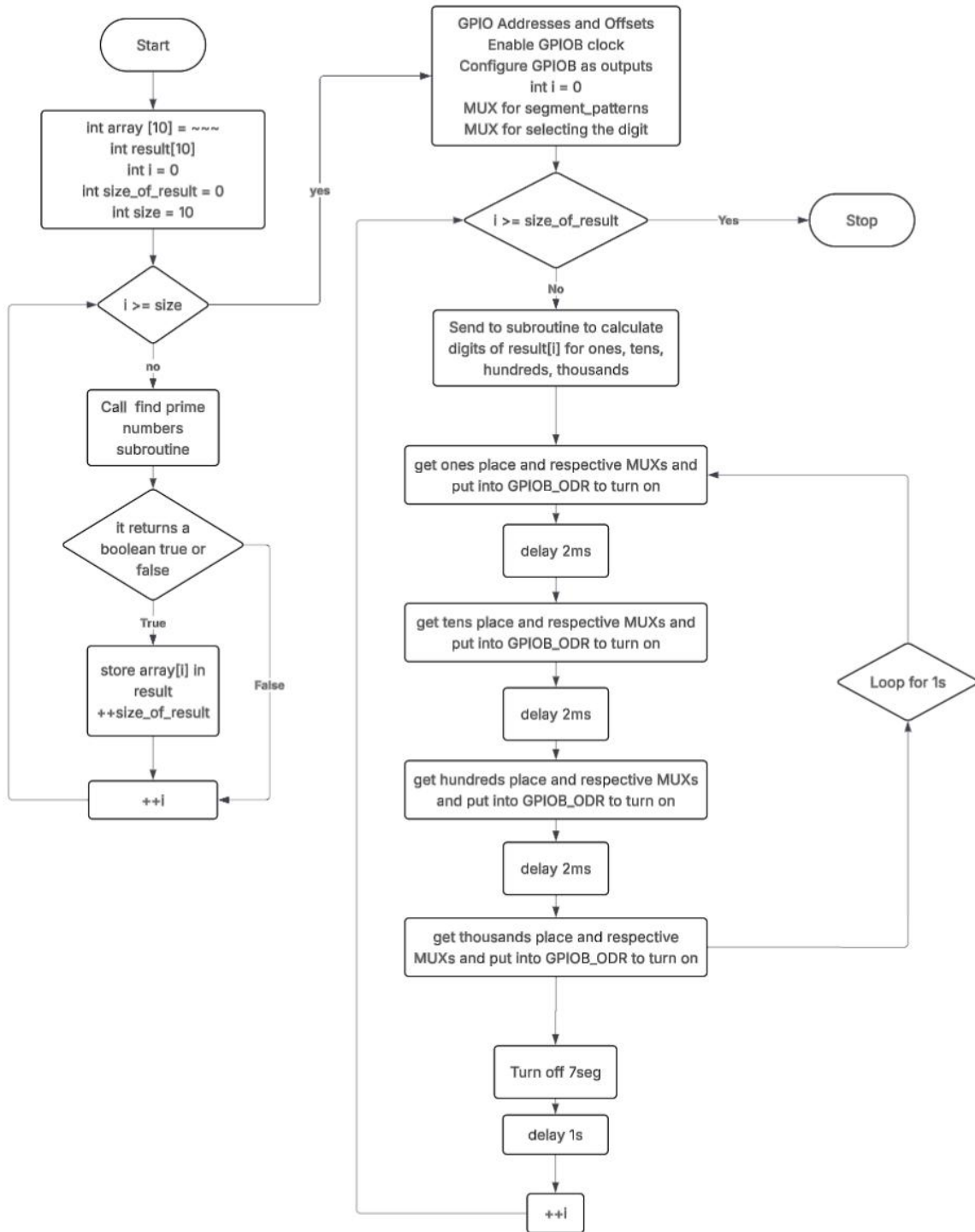
Memory Window:

Address: 0x20000000

0x20000000: 03 00 00 00 05 00 00 00 07 00 00 00 17 00
0x2000000E: 00 00 61 00 00 00 00 00 00 00 00 00 00

Task 3

Flowchart



Stack frames

Is prime subroutine

SP + 28	Result space. Pushed from R1 in main
SP + 24	Input parameter. Pushed from R0 in main
SP + 20	LR
SP + 16	R8
SP + 12	R7
SP + 8	R6
SP + 4	R5
SP + 0	R4 (stack pointer is here during the isPrime subroutine)

Stack is cleaned after getting all the prime numbers

Calc digits

SP + 8	LR
SP + 4	R5
SP + 0	R4

Display Number

SP + 36	R3
SP + 32	R2
SP + 28	R1
SP + 24	R0
SP + 20	LR
SP + 16	R8
SP + 12	R7
SP + 8	R6
SP + 4	R5
SP + 0	R4

Delay 2ms

SP + 44	R3
SP + 40	R2
SP + 36	R1
SP + 32	R0
SP + 28	LR
SP + 24	R8
SP + 20	R7
SP + 16	R6
SP + 12	R5
SP + 8	R4
SP + 4	R5
SP + 0	R4

Stack cleaned after running display_number subroutine

Delay 1 sec

SP + 20	R3
SP + 16	R2
SP + 12	R1
SP + 8	R0
SP + 4	R5
SP + 0	R4

Code:

ng_2026\microprocessors\ece2510_final_project\final_project.uvprojx - µVision [Non-Commercial Use License]

```
1      AREA      myData, DATA, READWRITE
2      GPIO_B_BASE EQU      0x48000400
3      RCC_BASE    EQU      0x40021000
4
5      GPIO_MODER   EQU      0x00
6      GPIO_IDR     EQU      0x10
7      GPIO_ODR     EQU      0x14
8
9      RCC_AHB2ENR  EQU      0x4C
10
11     result        SPACE    40          ; where the result array of primes is going to be stored
12     size_of_result DCD      0x00      ; to increase as a counter
13
14     ;
15     ;
16     seg_patterns  DCB      0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F
17     ALIGN
18     digit_select  DCW      0x0E00      ; Thousands
19     DCW           0x0D00      ; Hundreds (CA2)
20     DCW           0x0B00      ; Tens (CA3)
21     DCW           0x0700      ; Ones (CA4)
22     ALIGN
23     array         DCD      3, 5, 7, 9999, 20, 23, 100, 97, 98, 90
24     size          EQU      10
25
26     AREA      myCode, CODE, READONLY
27     EXPORT    __main
28     ENTRY
29
30     enable_clock  PROC
31
32         LDR     r2, =RCC_BASE
33         LDR     r1, [r2, #RCC_AHB2ENR]
34         ORR     r1, r1, #0x06          ; 0110 (bit 1 & bit 2)
35         STR     r1, [r2, #RCC_AHB2ENR]
36         BX     lr
37
38     ENDP
39
40     setup_gpio    PROC
41
42         ;      configuring port b 12 pins as output
43         LDR     r0, =GPIO_B_BASE
44         ;      ;
```

```

task2.s task3.s startup_stm321476xx.s system_stm3214xx.c
40 setup_gpio      proc
41
42                ;      configuring port b 12 pins as output
43                ldr     r0, =GPIO_B_BASE
44                ldr     r1, [r0, #GPIO_MODER]
45                ldr     r2, =0x00FFFFFF
46                bic     r1, r1, r2
47                ldr     r2, =0x00555555
48                orr     r1, r1, r2
49                str     r1, [r0, #GPIO_MODER]
50
51                bx      lr
52            endp
53
54 __main          proc
55
56                bl      enable_clock
57                bl      setup_gpio
58
59 ;      get prime numbers from the array into the result array
60 ; -----
61                mov     r0, #0            ; n
62                mov     r1, #0            ; true or false
63                mov     r2, #0            ; counter i = 0
64                ldr     r4, =size_of_result
65                ldr     r5, =0
66                str     r5, [r4]
67                ldr     r12, =array
68                ldr     r11, =result
69 loop
70                ldr     r0, [r12, r2, LSL#2] ; load array value into r0, register offset 4 bytes
71                push    {r0, r1}
72                bl      isPrime
73                pop     {r0, r1}          ; R1 now holds if true or false
74
75                cmp     r1, #1
76                beq     true              ; if true store
77                b       false
78 true            str     r0, [r11], #4
79                ldr     r5, [r4]
80                add     r5, r5, #1
81                str     r5, [r4]
82 false

```

```
task2.s task3.s startup_stm32l476xx.s system_stm32l4xx.c
79         ldr    r5, [r4]
80         add    r5, r5, #1
81         str    r5, [r4]
82 false
83         add    r2, #1 ; ++i
84         cmp    r2, #10
85         blt    loop
86
87
88         ldr    r12, [r4]
89
90 ;   START DISPLAYING
91 ; -----
92
93 reset_loop    ldr    r4, =result
94              mov    r5, #0
95 display_loop
96              cmp    r5, r12
97              bge    reset_loop
98
99              ; get the number from the array into R1
100
101              ldr    r0, [r4, r5, lsl #2] ; r2 holds the current number
102
103              bl    calc_digits ; process r2, r3 & r4
104
105              ; allocating 4 words
106              sub    sp, sp, #16 ; pushing the values to display
107
108              str    r0, [sp, #0] ; +0 = Thousands
109              str    r1, [sp, #4] ; +4 = Hundreds
110              str    r2, [sp, #8] ; +8 = Tens
111              str    r3, [sp, #12] ; +12 = Ones
112
113              ldr    r6, =75
114              ; (Array location) as arg to R0
115 mux_lsec    mov    r0, sp
116              bl    display_number
117              subs   r6, r6, #1
118              bne    mux_lsec ; turn on
119
120              bl    display_blank ;
121              bl    delay_lsec ; turn off
122
```

```

task2.s task3.s startup_stm321476xx.s system_stm3214xx.c
115 mux_lsec      mov     r0, sp
116              bl      display_number
117              subs    r6, r6, #1
118              bne     mux_lsec      ; turn on
119
120              bl      display_blank
121              bl      delay_lsec    ; turn off
122
123              add     sp, sp, #16
124
125              add     r5, r5, #1
126              b       display_loop
127              endp
128
129 display_blank  proc
130              ldr     r0, =GPIO_B_BASE
131              ldr     r1, =0x00000FFF
132              ldr     r2, [r0, #GPIO_ODR]
133              bic     r2, r2, r1
134              ldr     r1, =0x00000F00
135              orr     r2, r2, r1
136              str     r2, [r0, #GPIO_ODR]
137              bx      lr
138              endp
139
140 display_number proc
141              push    {r4-r8, lr}
142              mov     r4, r0          ; copying digit pointer to r4
143              mov     r5, #0          ; r5 = loop index
144              ldr     r6, =GPIO_B_BASE
145              ldr     r7, =seg_patterns
146              ldr     r8, =digit_select
147
148
149 mux_loop      cmp     r5, #3
150              bgt     mux_finish
151
152              ldr     r0, [r4, r5, lsl #2] ; get digit
153              ldrb    r1, [r7, r0]        ; load segment pattern in position
154                                          ; of digit to display
155
156              ldrrh   r2, [r8, r5, lsl #1] ; load the digit select half word
157
158

```


Target_1

task2.s

task3.s

startup_stm321476xx.s

system_stm3214xx.c

```

148
149 mux_loop      cmp     r5, #3
150               bgt     mux_finish
151
152               ldr     r0, [r4, r5, lsl #2] ; get digit
153               ldrb    r1, [r7, r0]        ; load segment pattern in position
154                                           ; of digit to display
155
156               ldrh    r2, [r8, r5, lsl #1] ; load the digit select half word
157
158               ; combination of segment pattern and digit selection
159               orr     r1, r1, r2
160               ldr     r3, =0x00000FFF
161               ldr     r0, [r6, #GPIO_ODR]
162               bic     r0, r0, r3
163               orr     r0, r0, r1
164               str     r0, [r6, #GPIO_ODR]
165
166               bl      delay_2ms
167
168               ; turn off display
169               ldr     r3, =0x00000FFF
170               ldr     r0, [r6, #GPIO_ODR]
171               bic     r0, r0, r3
172               ldr     r3, =0x00000F00 ; turn off digits    1000    0000 0000
173               orr     r0, r0, r3 ; 4digits    8 segments
174               str     r3, [r6, #GPIO_ODR]
175
176
177               add     r5, r5, #1
178               b       mux_loop
179
180 mux_finish     pop     {r4-r8, pc}
181               endp
182
183 ; calc digits return by registers
184 ; r0 holding the number to display
185 calc_digits    proc
186               push    {r4, r5, lr}
187
188               mov     r4, r0 ; r4 will hold the ones at the end
189               mov     r0, #0 ; thousands
190               mov     r1, #0 ; hundreds counter
191

```

ST-Link

```
target_1
task2.s task3.s startup_stm321476xx.s system_stm3214xx.c
181         endp
182
183 ; calc digits return by registers
184 ; r0 holding the number to display
185 calc_digits    proc
186         push    {r4, r5, lr}
187
188         mov     r4, r0          ; r4 will hold the ones at the end
189         mov     r0, #0          ; thousands
190         mov     r1, #0          ; hundreds counter
191         mov     r2, #0          ; tens counter
192         ldr     r5, =1000
193
194 calc_1000s     cmp     r4, r5
195                 blt     calc_100s
196                 sub     r4, r4, r5
197                 add     r0, r0, #1
198                 b       calc_1000s
199
200 calc_100s      cmp     r4, #100
201                 blt     calc_10s
202                 sub     r4, r4, #100
203                 add     r1, r1, #1          ; r4 storing hundreds
204                 b       calc_100s
205
206 calc_10s       cmp     r4, #10
207                 blt     done_calc
208                 sub     r4, r4, #10
209                 add     r2, r2, #1          ; r3 storing tens
210                 b       calc_10s
211
212 done_calc     mov     r3, r4
213                 pop     {r4, r5, pc}
214         endp
215
216
217 ; isPrime Function
218 ; -----
219 isPrime       PROC
220                 push    {r4-r8, lr}      ; sp drops by 24
221
222                 ldr     r4, [sp, #24]    ; load n from stack
223
224                 ...
```

```

task2.s task3.s startup_stm321476xx.s system_stm3214xx.c
214         endp
215
216
217 ; isPrime Function
218 ; -----
219 isPrime    PROC
220             push    {r4-r8, lr}    ; sp drops by 24
221
222             ldr     r4, [sp, #24]    ; load n from stack
223
224             ldr     r5, =2           ;
225             cmp     r4, r5           ; if n < 2 return false
226             blt     return_false    ;
227
228 ; Sqrt Function
229 ; -----
230
231 calc_sqrt  mov     r6, #0
232             mul     r7, r6, r6
233             cmp     r7, r4
234             bgt     sqrt_found
235
236             add     r6, r6, #1
237             b       calc_sqrt
238 sqrt_found sub     r6, r6, #1
239
240
241 prime_loop
242             cmp     r5, r6           ; r5 = i (counter) r6 = sqrt(n) returned by sqrt function
243             bgt     return_true      ; if at the end of loop, return true
244
245             udiv    r7, r4, r5       ; r7 = quotient = n / i
246             mul     r8, r7, r5       ; r8 = product = quotient * i
247
248             cmp     r8, r4           ; if product = n return false
249             beq     return_false
250
251             add     r5, r5, #1       ; i += 1
252             b       prime_loop
253
254
255 return_true ldr     r1, =1
256             str     r1, [sp, #28]
257

```

ST-Link Debugger

Contributions:

Avery Wang: Full Task 2, Work on Task 3, Flowchart of task 3

Sebastian De Leon: Full Task 1, work on Task 3, Stack frame of task 3